

Trabajo Integrador de PCSE

Sistema de riego automático

Carrera de Especialización en Sistemas Embebidos

Protocolos de Comunicación en Sistemas Embebidos

Alumno:

- **Esp. Ing. Gerardo Alexis Vilcamiza Espinoza**

Docente:

- **Ing. Pavelek Israel**

Colaboradores:

- **Dr. Ing. Pablo Gomez**
- **Mg. Ing. Gonzalo E. Sanchez**

Índice

- 1. Descripción de la idea a desarrollar**
- 2. Diagrama de bloques**
- 3. Estructura de código**
- 4. Drivers implementados**
- 5. Configuraciones de los módulos utilizados**
- 6. Conclusiones**

1. Descripción de la idea

Para este proyecto se plantea el desarrollo de un sistema de riego automático, basado en la placa NUCLEO-F411RE y diseñado con el objetivo de optimizar el uso del agua y asegurar el nivel adecuado de humedad en el suelo de una zona de cultivo. El sistema adapta su funcionamiento según el momento del día (día o noche) y cuenta con un mecanismo de seguridad que evita mojar a personas que puedan transitar durante el riego. Utiliza una electrobomba conectada a una red de mangueras y conductos, alimentada desde un tanque de 3 metros de altura. Asimismo, incorpora dos pantallas indicadoras de estado: un display LCD con módulo I2C PCF8574 ubicado en el campo y otra pantalla mediante UART ubicada en una central de control y monitoreo, permitiendo una supervisión remota y eficiente del sistema.

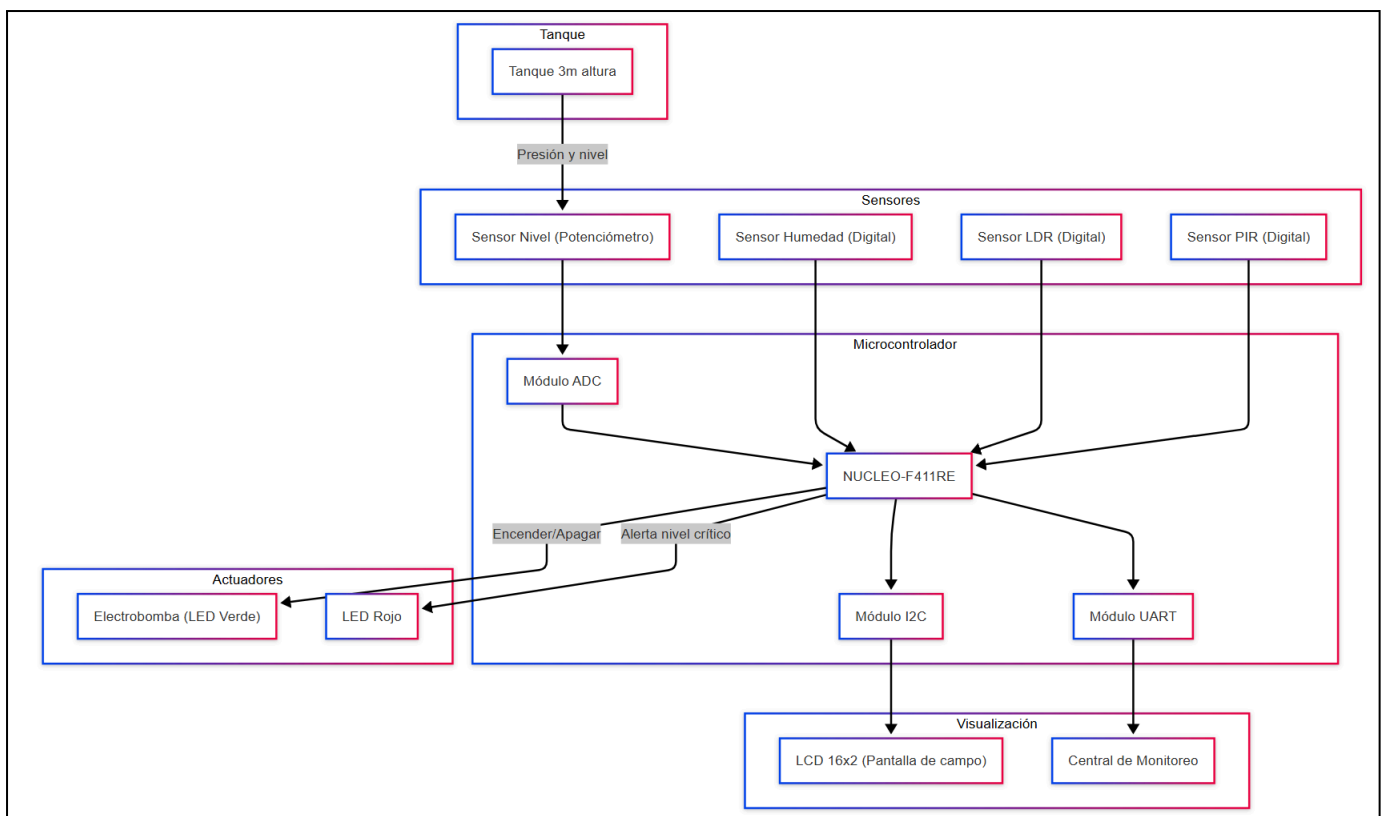
El sistema se activa automáticamente cuando un sensor de humedad de salida digital detecta que el suelo está seco. Dependiendo de la luz ambiental, determinada mediante un sensor LDR digital, se establece el tiempo de riego: si es de día, la bomba de agua se enciende durante 30 minutos, acompañada de un LED verde como indicador visual; si es de noche, el riego se reduce a 15 minutos. Durante el proceso de riego, si un sensor PIR detecta la presencia de una persona en las cercanías, la bomba se apaga de forma inmediata para evitar que la persona se moje, y se reactiva una vez que ya no se detecta movimiento. El sistema también supervisa el nivel de agua del tanque. Si este desciende por debajo del 20 % de su capacidad total, se muestra un mensaje de advertencia en la pantalla de la central de monitoreo. Si el nivel baja al 10 % o menos, la bomba se detiene automáticamente, y se genera una alerta tanto en la pantalla de la central como en la pantalla de campo. Adicionalmente, se enciende un LED rojo que indica la necesidad de rellenar el tanque. La bomba permanecerá desactivada hasta que el nivel de agua vuelva a ser suficiente. El nivel del agua se mide mediante un sensor de nivel ubicado en la parte superior del tanque, el cual también permite calcular la presión del agua en metros de columna de agua (mH₂O), que se convierte posteriormente a pascales. Tanto el porcentaje de llenado del tanque como el valor de la presión se visualizan en ambas pantallas del sistema.

Para simplificar el prototipado del proyecto, se emplearán pulsadores para simular las entradas digitales correspondientes al sensor de humedad, al sensor de nivel de agua y al sensor LDR, dado que en la práctica todos estos sensores pueden configurarse para entregar señales digitales. Adicionalmente, para emular el comportamiento del sensor de nivel del agua, se utilizará un potenciómetro conectado al módulo ADC, permitiendo así representar de forma analógica los distintos niveles del tanque durante las pruebas. Esta estrategia representa una buena práctica en el desarrollo de sistemas embebidos, ya que permite desacoplar

o aislar la programación de la lógica respecto al hardware físico, el cual no siempre se encuentra disponible en las etapas iniciales de diseño o validación.



2. Diagrama de bloques



3. Estructura del código

Inicialización y configuración

- `main()`
 - Llama a `HAL_Init()` y `SystemClock_Config()`
 - Entra en bucle infinito que invoca `loop()`
- `iniciarSistema()`
 - Inicializa módulos ADC, UART, GPIO, I2C y delays
 - Limpia y configura LCD
 - Envía mensajes de bienvenida por UART

Máquina de estados

- `loop()` usa `switch(estadoActual)` para despachar al handler correspondiente
- Breve `HAL_Delay(100)` en cada iteración
- Estados definidos en `enum estado_t`:
 1. ESTADO_INICIO
 2. ESTADO_MONITOREO
 3. ESTADO_RIEGO
 4. ESTADO_PAUSA
 5. ESTADO_ALERTA_BAJA
 6. ESTADO_ALERTA_CRITICA

State Handlers

- ESTADO_MONITOREO
 - Lee sensores (humedad, PIR, LDR, ADC)
 - Actualiza UART y LCD con valores actuales
 - Evalúa condiciones para transitar a riego o alertas
- ESTADO_RIEGO
 - Controla cuenta regresiva con `delayRead/delayWrite`
 - Supervisa PIR y niveles bajo/crítico, genera transiciones
 - Llama internamente al Monitor Handler para refrescar pantalla
- ESTADO_PAUSA

- Detiene bomba ante presencia, espera ausencia para reanudar
- Restaura contador de riego y llama a ESTADO_MONITOREO
- ESTADO_ALERTA_BAJA
 - Indica nivel bajo sin detener riego, muestra mensaje de alerta
 - Revisa recuperación de nivel para volver al handler de riego
- ESTADO_ALERTA_CRITICA
 - Detiene bomba, muestra alerta crítica en UART/LCD, enciende LED rojo
 - Al restaurarse suficiente nivel, apaga alerta y reanuda ESTADO_RIEGO

Interfaz de usuario

- UART: `sprintf` + secuencias ANSI para posicionar cursor, `uartSendString()`
- LCD I2C: `LCD_SetCursor()` + `LCD_PrintString()` con `lcdBuffer`

Configuración de reloj y manejo de errores

- `SystemClock_Config()`: arranque HSI, PLL y divisores AHB/APB
- `Error_Handler()`: bucle infinito tras fallos críticos de configuración

4. Drivers implementados

GPIO:

- GPIO_Init
 - Configura pines PB0/PB1 como salidas push-pull (LED bomba y LED alerta).
 - Configura pines PA1, PA4, PA7 como entradas pull-up (pulsadores humedad., PIR, LDR).
 - Inicializa un delay para antirrebote (50 ms).
- LED_Bomba_On / LED_Bomba_Off
 - Enciende o apaga PB1.

- Leer_Pulsador_Humedad / PIR / LDR
 - Lectura con HAL_GPIO_ReadPin, detecta flanco de subida, aplica antirrebote y alterna el estado flip-flop interno.

Timer:

- Timer_Start
 - Guarda el tick de inicio en la variable apuntada.
- Timer_Update
 - Cada segundo (si han pasado ≥ 1000 ms), decrementa *time y reinicia la marca de inicio.
 - Retorna true cuando el contador llega a 0.

API_delay:

- delayInit
 - Inicializa la estructura con la duración deseada y marca "no running".
- delayRead
 - Si no está corriendo, guarda el tick actual y arranca el conteo.
 - Si ya corría y ha transcurrido $\geq$ duration, resetea "running" y retorna true.
- delayWrite
 - Actualiza la duración del delay.
- delayIsRunning
 - Retorna el estado interno running.

API_ADC:

- ADC_Init
 - Habilita el reloj de ADC1.
 - Configura resolución a 12 bits, prescaler PCLK/4, modo single-conversion, canal 0, sample time 3 ciclos.
 - Llama a HAL_ADC_Init y HAL_ADC_ConfigChannel; si falla, invoca Error_Handler.
- ADC_Read

- Inicia conversión con HAL_ADC_Start, espera fin con HAL_ADC_PollForConversion, lee valor con HAL_ADC_GetValue y detiene el ADC.

LCD_I2C:

LCD_I2C.c

- LCD_Clear
 - Envía el comando de borrado (0x01).
- LCD_SetCursor
 - Calcula el offset según la fila (0 ó 0x40) y posiciona el cursor en la dirección adecuada.
- LCD_PrintString
 - Recorre la cadena carácter a carácter y los envía en modo datos.
- LCD_CreateChar
 - Define un carácter personalizado en la CGRAM: selecciona la posición y escribe los 8 bytes del mapa.

LCD_I2C_port.c

- I2C_Init
 - Habilita el reloj de I2C1 y configura hi2c1 (100 kHz, 7-bit, sin stretch).
- LCD_ExpanderWrite
 - Transmite un byte al módulo expensor del LCD por I²C.
- LCD_Write4Bits
 - Envía un seminibble gestionando señales de enable/backlight para latch.
- LCD_Send
 - Divide un byte en high/low nibble y los envía con LCD_Write4Bits.
- LCD_I2C_Init

- Secuencia de inicio: backlight, 0x30×3, cambio a 4-bits, comando 0x28 (2 líneas, 5×8), display off, clear, entry mode, display on.

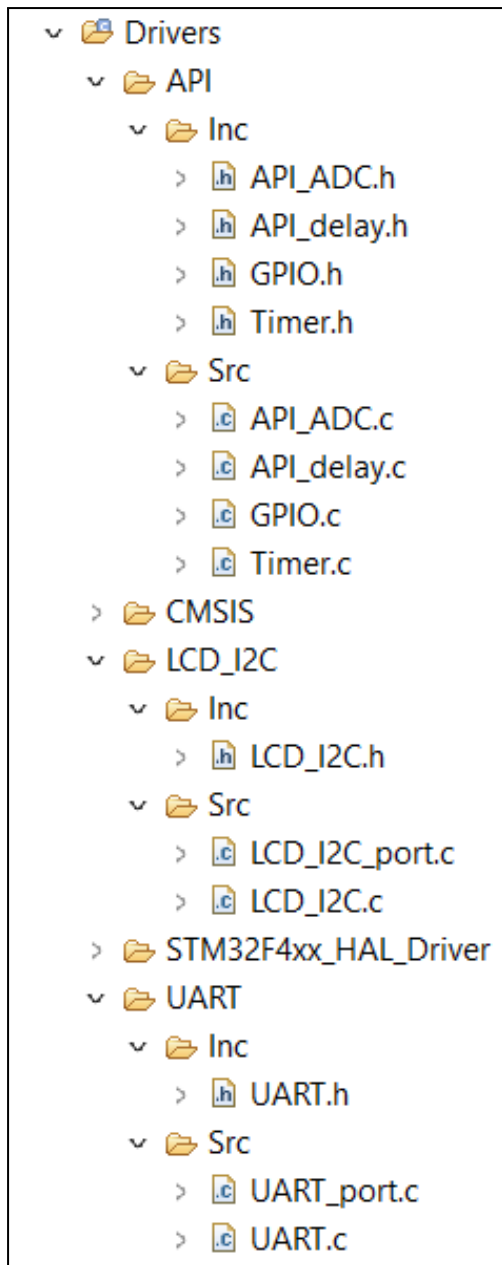
UART:

UART.c

- UART_Terminal_Init
 - Inicializa la interfaz UART para la terminal.
 - Limpia la pantalla con secuencias ANSI.
 - Muestra mensajes de bienvenida
 - Devuelve true si todo se inicializa correctamente.
- uartSendString
 - Copia la cadena al buffer interno si no es NULL y su longitud es menor al tamaño del buffer.
 - Transmite el buffer por UART usando HAL_UART_Transmit.

UART_port.c

- UART_Init
 - Habilita el reloj de USART2.
 - Configura huart2 con baudrate 115200, 8 bits, 1 stop bit, sin paridad, modo TX/RX, oversampling ×16.
 - Llama a HAL_UART_Init; retorna false si falla.
- uartSendStringSize
 - Copia exactamente size bytes al buffer TX y los transmite si $\text{size} \leq \text{buffer}$.
- uartReceiveStringSize
 - Recibe size bytes por UART en un buffer RX interno y luego los copia a pstring.



Distribución de los archivos de Drivers

5. Configuraciones de los módulos

UART

- Instancia: USART2
- Baud Rate: 115200
- Word Length: 8 bits
- Stop Bits: 1
- Parity: None
- Modo: TX/RX
- Oversampling: ×16

I2C

- Instancia: I2C1
- Velocidad de reloj: 100 kHz
- Ciclo de trabajo: 2
- Addressing Mode: 7 bits
- DualAddressMode: deshabilitado
- General Call: deshabilitado
- Clock Stretching: deshabilitado

ADC

- Instancia: ADC1
- Resolución: 12 bits
- Prescaler: PCLK/4
- Sample Time: 3CYCLES
- Canal: A0

6. Conclusiones

El diseño del sistema de riego automático, basado en lógica adaptativa que regula el tiempo de riego según la luz ambiental (día/noche) y detiene la bomba ante detección de personas, demuestra en el prototipado funcional su potencial para optimizar el consumo de agua y mejorar la seguridad del usuario, aunque su desempeño en un entorno real está pendiente de validación

La arquitectura de software basada en una máquina de estados bien definida y en drivers modulares (GPIO, ADC, I2C, UART y temporizadores) facilita la mantenibilidad, la escalabilidad y la trazabilidad del código, permitiendo aislar comportamientos concretos (monitoreo, riego, pausas y alertas) y simplificar futuras ampliaciones o ajustes de parámetros

La estrategia de prototipado mediante pulsadores para simular sensores digitales y el uso de un potenciómetro conectado al ADC para emular el nivel de agua representan una buena práctica de desacoplamiento entre la lógica de control y el hardware, garantizando validaciones tempranas y robustez en entornos de prueba, y sentando las bases para una integración posterior con sensores reales o sistemas de comunicación remota

.